

Banking System (OO)

Problem Statement

Implement accounts with create, deposit, withdraw, transfer, and top-K transactions.

Variants

- Overdraft rules; multi-currency

- Per

How to Leverage the Solution

Centralize validation in Bank; extend Transaction indexing for richer queries.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;
import java.util.concurrent.atomic.AtomicLong;

public final class Bank {
    private final Map<Long, Account> accounts = new HashMap<>();
    private final List<Transaction> allTransactions = new ArrayList<>();
    private final AtomicLong nextId = new AtomicLong(1L);

    public synchronized long createAccount(String owner, long initialCents) {
        if (initialCents < 0) throw new IllegalArgumentException("Negative initial balance");
        long id = nextId.getAndIncrement();
        accounts.put(id, new Account(id, owner, initialCents));
        if (initialCents > 0) allTransactions.add(Transaction.deposit(id, initialCents));
        return id;
    }

    public synchronized void deposit(long accountId, long cents) {
        requirePositive(cents); Account acc = get(accountId); acc.balanceCents += cents; allTransactions.
add(Transaction.deposit(accountId, cents));
    }

    public synchronized void withdraw(long accountId, long cents) {
        requirePositive(cents); Account acc = get(accountId); if (acc.balanceCents < cents) throw new Ill
egalStateException("Insufficient funds");
        acc.balanceCents -= cents; allTransactions.add(Transaction.withdraw(accountId, cents));
    }

    public synchronized void transfer(long fromId, long toId, long cents) {
        requirePositive(cents); Account from = get(fromId); Account to = get(toId); if (from.balanceCents
< cents) throw new IllegalStateException("Insufficient funds");
        from.balanceCents -= cents; to.balanceCents += cents; allTransactions.add(Transaction.transfer(fr
omId, toId, cents));
    }

    public synchronized long balance(long accountId) { return get(accountId).balanceCents; }
    public synchronized List<Transaction> topKByAmount(int k) {
        PriorityQueue<Transaction> pq = new PriorityQueue<>(Comparator.comparingLong(t -> t.amountCents))
;
        for (Transaction t : allTransactions) {
            if (pq.size() < k) pq.offer(t);
            else if (t.amountCents > pq.peek().amountCents) { pq.poll(); pq.offer(t); }
        }
        List<Transaction> result = new ArrayList<>(pq); result.sort((a, b) -> Long.compare(b.amountCents,
a.amountCents)); return result;
    }

    private Account get(long id) { Account acc = accounts.get(id); if (acc == null) throw new NoSuchEl
```

Banking System (OO)

```
ementException("No account " + id); return acc; }
    private static void requirePositive(long cents) { if (cents <= 0) throw new IllegalArgumentException("Non-positive amount"); }
    public static final class Account { public final long id; public final String owner; private long balanceCents; Account(long id, String owner, long balanceCents) { this.id = id; this.owner = owner; this.balanceCents = balanceCents; } }
    public static final class Transaction {
        public enum Type { DEPOSIT, WITHDRAW, TRANSFER }
        public final Type type; public final long fromAccountId; public final long toAccountId; public final long amountCents; public final long timestampMs;
        private Transaction(Type type, long fromAccountId, long toAccountId, long amountCents) { this.type = type; this.fromAccountId = fromAccountId; this.toAccountId = toAccountId; this.amountCents = amountCents; this.timestampMs = System.currentTimeMillis(); }
        public static Transaction deposit(long toId, long amount) { return new Transaction(Type.DEPOSIT, -1, toId, amount); }
        public static Transaction withdraw(long fromId, long amount) { return new Transaction(Type.WITHDRAW, fromId, -1, amount); }
        public static Transaction transfer(long fromId, long toId, long amount) { return new Transaction(Type.TRANSFER, fromId, toId, amount); }
    }
}
```